Vibe-Coded Is the New "Made in China"

Jan 10, 2026 7 min read

TL;DR

"Vibe-coded" is becoming a stigma label, like "Made in China" once was. The code itself is often fine. The distrust comes from what the label signals: low investment, no skin in the game, likely abandonment. Open source historically worked because building something took so much effort that releasing it meant you cared. When anyone can scaffold an app in hours, that trust model breaks. We are still figuring out how to work alongside AI. Until then, the skepticism makes sense.

Yesterday I was browsing r/selfhosted (a subreddit for self-hosted software enthusiasts) and noticed something: there are far more tools and projects being released than a year ago, but many are met with immediate hostility. If they're labeled "vibe-coded", comment sections fill with skepticism.

But why? The code often works fine. Sometimes it's even better than what a solo developer would write manually. So what's the actual problem?

What is vibe-coding?

Vibe coding is a term coined by Andrej Karpathy in early 2025.[1] He described it as a way of coding where you "fully give in to the vibes, embrace exponentials, and forget that the code even exists" — letting LLMs handle the implementation while you focus on what you want to build, not how to build it.

When Difficulty Was the Filter

Before AI-assisted coding existed, open source developers who released a project had already invested enormous amounts of time just to reach the point of publishing. They had "skin in the game". The implicit assumption was that they would keep maintaining their creation into the foreseeable future. You had to really stand behind an idea to take it that far. You had to be a certain kind of person.

That assumption held because building software was hard. Grinding through documentation, debugging obscure errors, learning new concepts just to implement a feature. All of that created a barrier. If you made it through, you probably cared enough to stick around. **The difficulty was a filter.**

Now the barrier is gone. Everyone can scaffold an application in hours that would have taken months. More importantly, **everyone can code above their skill level.** You don't need to understand the domain deeply. You don't need to know why your architecture works. You just need a vision and enough prompting skill to get something functional. This creates scenarios where project authors might not understand the code they're releasing. They can't fix edge cases because they don't know how the system actually works. They can't extend features because they didn't design the foundation. They can't maintain the project long-term because they have no "emotional attachment" to something that took an afternoon to build.

The community senses this[2]. The distrust isn't *just* about code quality. It's also about projected longevity.

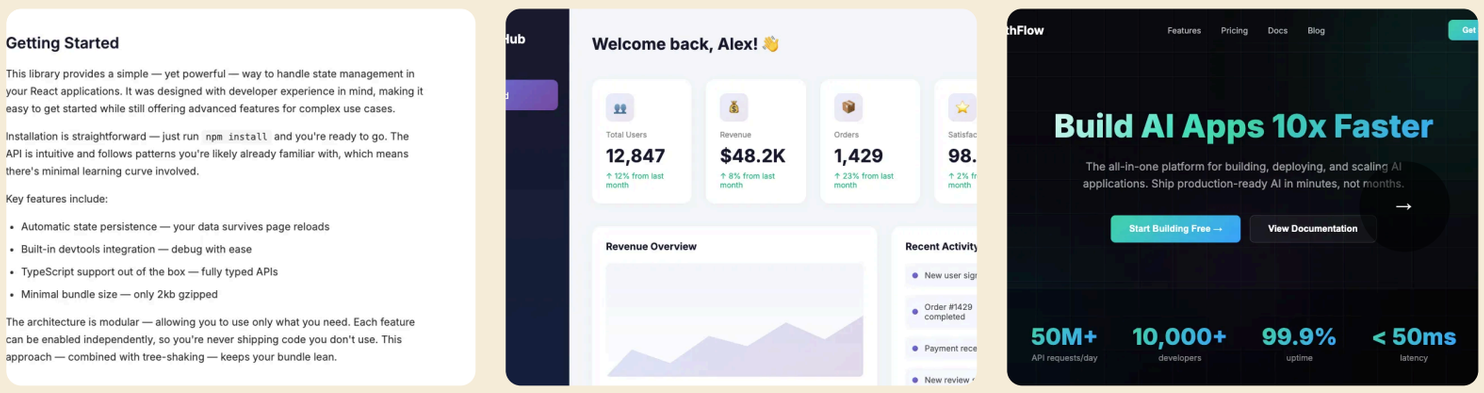
The Stigma of a Label

Remember when "Made in China" was shorthand for cheap, disposable junk? The label carried instant assumptions about quality, durability, and care. It didn't matter if a specific product was actually well-made. The label triggered the assumption.

Today, China produces everything from dollar-store toys to premium electronics. The stigma took decades to shake, and for many people it never fully did. The label still carries weight even when the reality has changed.

"Vibe-coded" is becoming the same kind of marker. When someone admits their project was vibe-coded, they're signaling: "I didn't grind through the learning curve. I might not fully understand how this works." Whether that's actually true for any given project doesn't matter. The label triggers the assumption. And the assumption is: this will be abandoned within six months.

It doesn't help that AI output has a recognizable character.[3] In tone, in style, in everything it produces. You've seen it: the writing heavy on em-dashes, the UI designs converging on the same purple gradients and card layouts, the documentation that sounds helpful but somehow hollow.



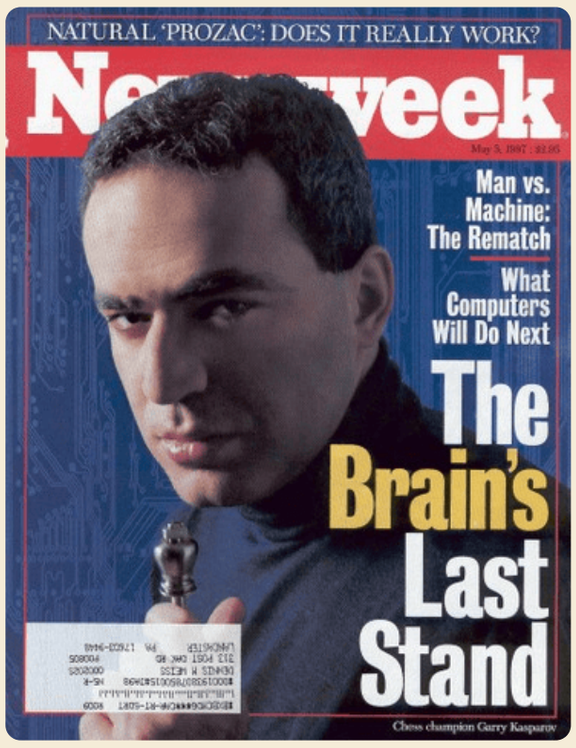
The recognizable aesthetic of AI-generated output (of course I generated these examples)

AI output often lands in an uncanny valley: close enough to human work to pass at first glance, different enough to feel *off* when you look closer. When people sense they're consuming AI-generated content, it triggers a specific frustration: *why am I giving my attention to something the author didn't care enough to create themselves?* It feels like a waste of time. Like being tricked into reading someone's homework that ChatGPT wrote.

We've Been Here Before

When Deep Blue beat Kasparov in 1997, many assumed chess was finished.[4] Why care about a game that computers had "solved"? But chess is more popular today than ever. Grandmasters train with engines, analyze positions with AI, and stream to millions of viewers who want to watch *humans* compete.

The key shift: humans stopped fighting computers and started working alongside them.[5,6]



Kasparov vs Deep Blue, 1997 — the moment humans realized machines were catching up (Source: www.warpnews.org)

We're not there yet with coding. Right now, it feels like competition. Hand-crafted projects versus an endless flood of AI-generated slop. The instinctive reaction is rejection. Distrust anything that

smells like vibe-code. And honestly? The reaction isn't irrational. If chess is any guide, this is a phase. Eventually we'll figure out how to harness the power of AI while still maintaining the human touch. But we haven't found that equilibrium yet. And until we do, the skepticism is a reasonable defense mechanism.

The Ones Who Get It Seemingly Right

But blanket distrust is too blunt. Some of the most respected voices in software development are already finding ways to use AI that don't trigger the alarm bells.

DHH, the creator of Ruby on Rails and someone famously opinionated about code aesthetics, put it bluntly: "You can't let the slop and cringe deny you the wonder of AI. This is the most exciting thing we've made computers do since we connected them to the internet."[7]

Tanner Linsley, creator of TanStack (React Query, TanStack Router, and other beloved frontend libraries), was asked if he uses AI agents to write his tools. His answer: "In a small percentage and responsibly, yes."[8] The key word is *responsibly*. He's not handing off architecture decisions. He's augmenting his workflow while staying in control.

And then there's Boris Cherny, creator of Claude Code. When someone asked if he writes any code himself, he replied: "In the last thirty days, 100% of my contributions to Claude Code were written by Claude Code."[9] This isn't someone blindly accepting AI output. Cherny built the tool from scratch, understands every architectural decision, and reviews everything that goes in. The result? Claude Code has become one of the most widely adopted AI coding tools in the industry.

The pattern: developers who care deeply about quality aren't rejecting AI. They're integrating it carefully, staying the author while letting AI handle the grunt work.

My last blog post took me a week to write, even though I used AI heavily throughout. Without AI, the result would have been worse English, missing arguments, and weaker examples. Was the post bad because AI helped? I'd argue no, but you're the judge 😉.

Where This Leaves Us

Vibe-coding isn't inherently bad. It's a tool that dramatically lowers the barrier to creating software. That democratization has real value. People who couldn't build things before can build things now. Problems that weren't worth solving manually become tractable when AI handles the tedious parts.

The good news: time still works as a filter. A project that's been actively maintained for two years, with real users and resolved issues, demonstrates commitment that can't be faked in an afternoon. But at the point of first contact, when a new project appears with a polished README and no track record, the only information available is what the author discloses. And "I vibe-coded this" triggers alarm bells.

I suspect we'll start valuing longevity even more. Projects that demonstrate understanding through thoughtful issue responses will build trust that flashy READMEs can't buy. We're in an awkward transition. The old trust model assumed creation difficulty as proof of commitment. The new reality needs different proof. Until the community figures out what that proof looks like, vibe-coded projects will keep hitting skepticism.

That skepticism isn't the enemy. It's the ecosystem trying to protect itself while the rules are being rewritten.

1. https://x.com/karpathy/status/1886192184808149383 ↩
2. Stack Overflow's 2025 Developer Survey Reveals Trust in AI at an All Time Low: https://stackoverflow.co/company/press/archive/stack-overflow-2025-developer-survey/ ↩
3. https://www.nytimes.com/2025/12/03/magazine/chatbot-writing-style.html ↩
4. https://www.chess.com/blog/Chessable/human-vs-machine-kasparovs-legacy ↩
5. https://www.ted.com/talks/garry_kasparov_don_t_fear_intelligent_machines_work_with_them ↩

6. Kasparov, G. (2017). *Deep Thinking: Where Machine Intelligence Ends and Human Creativity Begins*. PublicAffairs. ↩

7. https://x.com/dhh/status/2007503687745490976 ↩

8. https://x.com/tannerlinsley/status/2009763002011328806 ↩

9. https://x.com/bcherny/status/2004897269674639461 ↩